

Unit 5



Unsupervised Learning- Overview, What Is Cluster Analysis, Different Types of Clustering's, Different Types of Clusters

K-means-The Basic K-means Algorithm, Additional Issues, Bisecting K-means, K-means and Different Types of Clusters, Strengths and Weaknesses, K-means as an Optimization Problem

Cluster Evaluation-Overview, Unsupervised Cluster Evaluation Using Cohesion and Separation, Unsupervised Cluster Evaluation Using the Proximity Matrix, Determining the Correct Number of Clusters, Supervised Measures of Cluster Validity, Assessing the Significance of Cluster Validity Measures, Choosing a Cluster Validity Measure



Unsupervised Learning- Overview.

- Type of Machine learning- No labeled output is provided, The algorithm tries to discover hidden patterns or structures in data.

Characteristics:

- No predefined classes or target values.
- Focuses on structure, similarity, grouping, or density.
- Mostly used for exploratory data analysis(EDA- is an approach to analyze data sets to discover patterns, trends, relationships, and anomalies using statistical summaries and visual methods)

- Cluster: A collection of data objects
 - similar (or related) to one another within the same group
 - dissimilar (or unrelated) to the objects in other groups
- Cluster Analysis is a technique that divides data into groups called clusters such that:
 - Objects within the same cluster are highly similar
 - Objects in different clusters are dissimilar

Clustering can be used either to understand data or to support efficient processing and decision-making.

Cluster analysis has played a crucial role in many disciplines, including:

- Psychology and Social Sciences
- Biology and Bioinformatics
- Statistics
- Pattern Recognition
- Information Retrieval
- Machine Learning
- Data Mining

Goals of Cluster Analysis:

Meaningful Clustering

- Clusters reflect the *natural structure* present in the data
- Used to gain insight and understanding

Utility-Based Clustering

- Used for data summarization
- Reduces data size while preserving important characteristics

Applications of Cluster Analysis:

Meaningful Clustering

- Discovering natural groupings in data
- Identifying hidden patterns and relationships
- Supporting exploratory data analysis (EDA)

Cluster analysis is widely applied in biology, information retrieval, climate science, psychology and medicine, and business to discover patterns, classifications, and meaningful groupings in large datasets.

Utility-Based Clustering

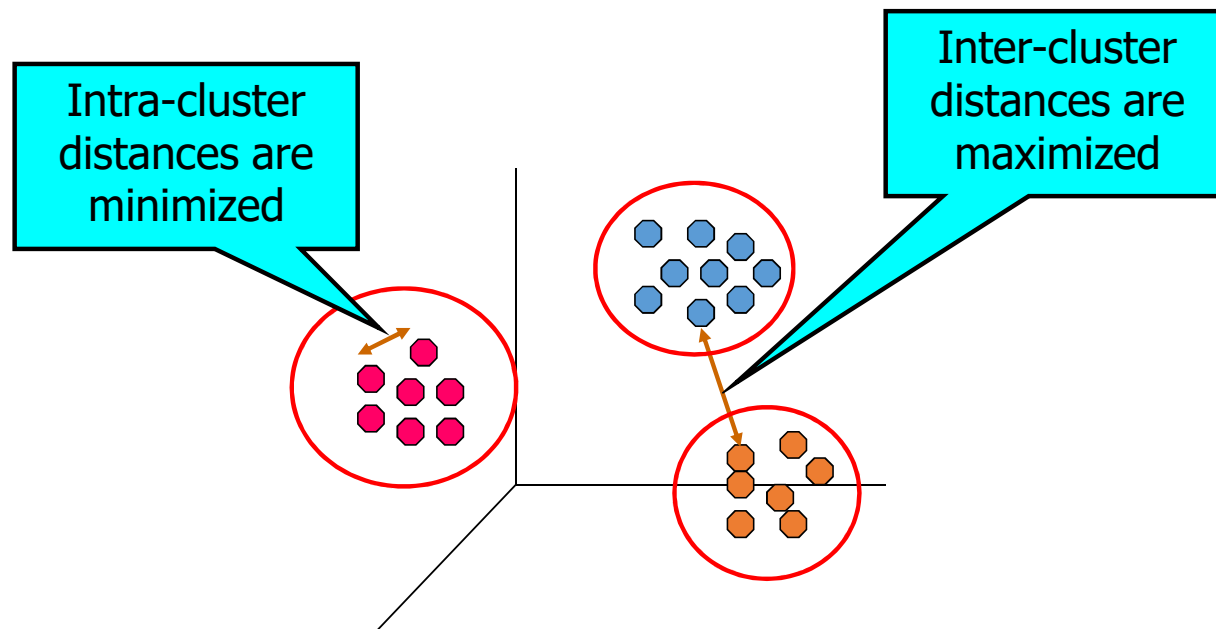
- Data compression and summarization
- Improving efficiency of search and retrieval
- Supporting downstream tasks such as classification

Utility-based clustering emphasizes performance and scalability rather than interpretability.

Cluster prototypes support data summarization by allowing complex algorithms to run on reduced datasets, enable compression through vector quantization, and improve nearest neighbor search efficiency by limiting distance computations to nearby clusters.

Definition of Cluster Analysis

- Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups



- A good clustering method will produce high quality clusters with
 - high intra-class similarity
 - low inter-class similarity
- The quality of a clustering result depends on both the similarity measure used by the method and its implementation
- The quality of a clustering method is also measured by its ability to discover some or all of the hidden patterns

Clusterings can be classified based on:

- Structure of clusters
- Membership of objects
- Coverage of data objects

Major Classifications

- Hierarchical vs Partitional
- Exclusive vs Overlapping vs Fuzzy
- Complete vs Partial



Hierarchical vs Partitional

Partitional Clustering

- Divides data into **non-overlapping clusters**
- Each object belongs to **exactly one cluster**
- No subclusters are allowed
- Example: **K-Means**
- Can be obtained by selecting one level of a hierarchy

Hierarchical Clustering

- Produces **nested clusters** organized as a **tree (dendrogram)**
- Root represents all objects
- Internal nodes represent unions of subclusters
- Leaves often represent individual data points
- Can be viewed as a **sequence of partitional clusterings**

A hierarchical clustering can be “cut” at any level to produce a partitional clustering.

Different types of Clustering

Exclusive vs Overlapping vs Fuzzy

- **Exclusive clustering:** Each object belongs to only one cluster
- **Overlapping (Non-exclusive) clustering:** Objects may belong to multiple clusters
- **Fuzzy / Probabilistic clustering:**
 - Each object belongs to all clusters with weights/probabilities (0–1)
 - Membership values sum to 1
 - Often converted to exclusive clustering by choosing the highest membership

Complete vs Partial Clustering

Complete clustering:

Every object is assigned to some cluster

Partial clustering:

Some objects (noise, outliers, background data) are left unclustered

Useful when only meaningful groups are of interest

Different types of Cluster

- Clustering groups similar data objects based on analysis goals.
- Different applications need different notions of similarity
- Example: Document browsing systems must ensure, all documents are included in some cluster
- Clustering can be: Based on distance, Based on density, Based on connectivity, Based on shared properties

Major Types of Clusters

- Well-Separated Clusters
- Prototype-Based (Center-Based) Clusters
- Graph-Based Clusters
- Density-Based Clusters
- Shared-Property (Conceptual) Cluster

Well Separated Cluster:

- A cluster where: Each object is closer to objects in the same cluster than to any object outside the cluster
- often uses a distance threshold

Key Characteristics:

- Clusters are far apart
- Minimal overlap
- Can be any shape (not necessarily circular)

Prototype-Based (Center-Based) Clusters:

Objects are closer to their cluster prototype than to prototypes of other clusters.

Type of Prototypes:

- Centroid → Mean of points (numerical data)
- Medoid → Most representative object (categorical data)
- Clusters tend to be globular (spherical) Widely used in practice
- Algorithms- K means



Graph-Based Clusters:

Data represented as a graph

- Nodes → objects
- Edges → connections/similarity
- Cluster = connected component
- Contiguity-Based Clusters: Objects connected if within a given distance
- Each object is close to some object in the cluster
- Strength: Handles irregular and intertwined shapes

Density-Based Clusters:

Cluster = dense region of points

Surrounded by low-density regions

Key Characteristics

- Handles: Noise, Outliers, Arbitrary shapes
- Avoids merging clusters through sparse bridges

Shared-Property (Conceptual) Clusters

- Cluster = set of objects sharing a common property
- Generalizes all previous cluster definitions
- Examples: Objects closest to the same centroid
- Intertwined geometric shapes
- Region-based or rule-based groupings
- Conceptual Clustering- requires domain-specific definitions and moves towards pattern recognition

Summary- Cluster Analysis

Type of Clustering Analysis	Description	Key Feature	Example Algorithms
Hierarchical	Produces a tree-like (nested) structure of clusters	No need to predefine number of clusters	Agglomerative, Divisive
Partitional	Divides data into non-overlapping clusters	Each object belongs to exactly one cluster	K-Means, K-Medoids
Exclusive	Each object belongs to only one cluster	Hard assignment	K-Means
Overlapping	Objects can belong to multiple clusters	Shared membership	Clique-based clustering
Fuzzy	Objects belong to clusters with degrees of membership	Soft assignment (0–1)	Fuzzy C-Means
Complete	All data objects are assigned to clusters	Full coverage	K-Means
Partial	Some objects remain unclustered	Handles noise/outliers	DBSCAN

Summary- Types of cluster

Type of Cluster	Definition	Shape	Noise Handling	Typical Use Case
Well-Separated	Objects are closer to each other than to any outside object	Any	Poor	Clearly distinct groups
Prototype-Based (Center-Based)	Objects are closest to a centroid or medoid	Globular	Moderate	Numeric data clustering
Graph-Based	Objects form connected components in a graph	Irregular	Poor	Network or relational data
Clique-Based	Objects form fully connected subgraphs	Globular	Poor	Tight-knit groups
Density-Based	Dense regions separated by sparse areas	Arbitrary	Strong	Noisy, real-world data
Shared-Property (Conceptual)	Objects share a common concept or property	Any	Depends	Pattern recognition

Prototype-Based Clustering Techniques

- Create a **one-level (flat) partitioning** of data-Data is split once/There is no parent–child relationship between clusters
- Each data object belongs to **exactly one cluster**
- Each cluster is represented by a **prototype**
- Goal: minimize distance between data points and their cluster prototype

Common Techniques

- **K-Means- The mean is the average value of a set of data points- the mean of a cluster is called its centroid.**
- **K-Medoids- The medoid is the actual data point whose average dissimilarity to all other points in the cluster is minimum.**

K-Means Clustering

- Prototype is the centroid
- Centroid = mean of all points in the cluster
- Works best with numerical data
- Uses Euclidean distance

Characteristics

- Fast and simple
- Sensitive to outliers.
- Centroid may not be an actual data point

$$X = \{2, 4, 10, 12\}$$

$$K = 2$$

1. Initialize the centroids: $C1 = 2, C2 = 10$
2. Assign data points to nearest cluster

Data Point	Distance to C1 (2)	Distance to C2 (10)	Assigned Cluster
2	0	8	Cluster 1
4	2	6	Cluster 1
10	8	0	Cluster 2
12	10	2	Cluster 2

4. Final Cluster:

Cluster 1: $\{2, 4\}$, centroid = 3

Cluster 2: $\{10, 12\}$, centroid = 11

3. Recompute centroids:

New Centroids

Cluster 1: mean of $\{2, 4\}$

$$C1 = (2 + 4)/2 = 3$$

Cluster 2: mean of $\{10, 12\}$

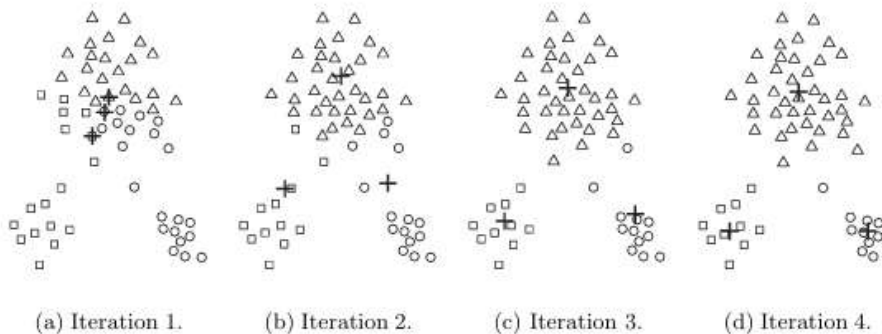
$$C2 = (10 + 12)/2 = 11$$

4. Check for Convergence:

- Reassign points $\rightarrow$ same clusters
- Algorithm converges

Algorithm 5.1 Basic K-means algorithm.

- 1: Select K points as initial centroids.
 - 2: **repeat**
 - 3: Form K clusters by assigning each point to its closest centroid.
 - 4: Recompute the centroid of each cluster.
 - 5: **until** Centroids do not change.
-



Assigning Points to Clusters

- In K-means, each data point is assigned to the closest centroid
- “Closest” must be quantified mathematically
- This is done using a proximity (**similarity or distance**) measure.
- Determines cluster shape
- Directly affects clustering accuracy

Choice of proximity measure depends on data type. Various proximity measures are:

- **Euclidean distance** -Most common for numeric, Euclidean data
- **Manhattan distance** -Alternative for Euclidean data
- **Cosine similarity**- Suitable for text and document data
- **Jaccard measure** -Used for sets and document similarity.

Simple Measures Are Preferred in K-Means

- Computational Considerations - K-means is an iterative algorithm
- In each iteration: Every point is compared with every centroid.
- Complex similarity measures → high computation cost

Other variant of K-means is Bisecting K-means

Centroids and Objective Functions:

In K-means, Step 4 is: Recompute the centroid of each cluster

- This step is needed because the centroid definition depends on Proximity (distance) measure and Clustering goal.

Further,

- A centroid is a representative point of a cluster.
- Its exact definition depends on- **Type of distance used and Objective of clustering.**

Examples: Mean → when minimizing squared Euclidean distance. Median or medoid → when using other distance measures

Objective Functions

- A mathematical expression that defines the goal of clustering.
- *This evaluates the quality of clusters*

Common K-means Objective

Minimize $\sum(\text{distance of each point to its closest centroid})^2$

- ✓ Encourages compact clusters
- ✓ Penalizes distant points

$$\text{SSE} = \sum_{x \in C} \text{dist}(x, c)^2$$

Sum of Square Error used in K-means

Data Points

(1, 1), (2, 3), (6, 2)

Centroid (already computed)

$c = (3, 2)$

Point 1: (1, 1)

$$\text{dist} = \sqrt{(1 - 3)^2 + (1 - 2)^2} = \sqrt{4 + 1} = \sqrt{5}$$

$$(\sqrt{5})^2 = 5$$

Calculation of SSE for Data in Euclidean Space

Consider the following 2D data points: (2,2), (4,4), (6,6), (8,8)

Determine

- Find the centroid of the cluster
- Compute the SSE
- Comment on the clustering quality

1
$$c = \left(\frac{2+4+6+8}{4}, \frac{2+4+6+8}{4} \right) = (5,5)$$

2

Point	Distance to (5,5)	Squared Distance
(2,2)	($\sqrt{18}$)	18
(4,4)	($\sqrt{2}$)	2
(6,6)	($\sqrt{2}$)	2
(8,8)	($\sqrt{18}$)	18

3
$$SSE = 18 + 2 + 2 + 18 = \boxed{40}$$

Quality of cluster:

1. Data points are **symmetrically distributed** when they are **evenly balanced around a central point (the centroid)** in all directions.
2. Centroid lies at the center.
3. SSE reflects spread around centroid.
4. K-means has achieved minimum SSE for this cluster

Calculation of SSE for Document Data

- Documents are not points in Euclidean space.
- Represented using a Document–Term Matrix
- Each document is a high-dimensional vector
- Distance-based measures like Euclidean distance are often not meaningful

Document–Term Matrix

- Rows → Documents
- Columns → Terms (words)
- Values → Term frequency / TF
- Each document is treated as a vector in high-dimensional space

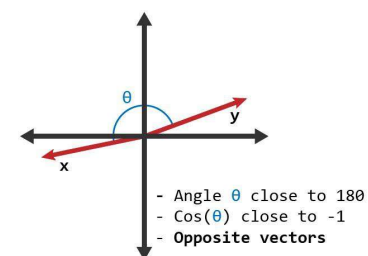
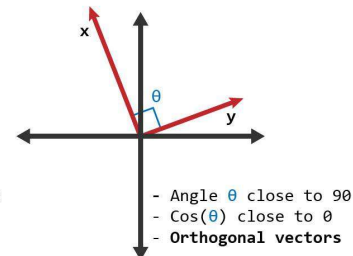
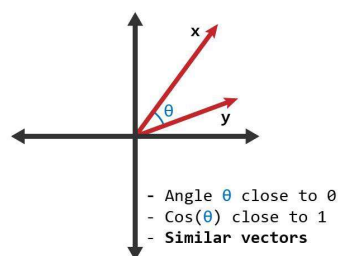
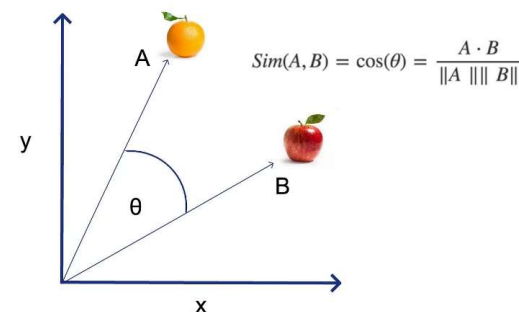
Calculation of SSE for Document Data

Cosine similarity $\cosine(x, c) = \frac{x \cdot c}{\|x\| \|c\|}$

Cohesion $\text{Total Cohesion} = \sum_{i=1}^K \sum_{x \in C_i} \cos(x, c_i)$

- Measures the **angle** between two vectors
- Ignores document length
- Focuses on **direction (content similarity)**

Cosine Similarity



maximize the similarity of the documents in a cluster to the cluster centroid; this quantity is known as the cohesion of the cluster.

Choosing initial Centroids

- Choosing the proper initial centroids is the key step of the basic K-means procedure.
- A common approach is to choose the initial centroids randomly, but the resulting clusters are often poor.
- K-means starts with an initial set of centroids.
- The algorithm then iteratively assigns points to nearest centroids
- Updates centroids to reduce SSE
- K-means is guaranteed to converge, but only to a local minimum

Different initial centroids → different final clusters → different SSE

Choosing initial Centroids

- Random initialization is unreliable, especially for complex datasets
- Because it can lead to: Poor cluster quality, High SSE and Suboptimal (local minimum) solutions

Approaches to choose initial centroids:

Method	Advantage	Limitation	Method explanation
Random	Simple	Poor quality	Initial centroids are chosen randomly from the dataset, making the method simple but highly sensitive to poor starting positions.
Multiple restarts	Better chance	Expensive	K-means is run several times with different random initial centroids, and the clustering with the minimum SSE is selected.
Hierarchical	Informed	Costly	Initial centroids are obtained from clusters formed using hierarchical clustering on a small sample of the data.
Farthest-point	Well separated	Outliers	Each new centroid is selected as the point farthest from all previously chosen centroids to ensure good separation.
Sampling + farthest	Efficient	Sampling bias	The farthest-point strategy is applied on a random sample of data to reduce computation and avoid outliers
Density-based	Robust	Extra computation	Centroids are selected from high-density regions of the data to better represent true clusters.
K-means++	Best trade-off	Slight overhead	Centroids are chosen probabilistically so that points far from existing centroids have a higher chance of being selected, improving cluster quality.

Choosing initial Centroids

- Newer Approach for initializing centroids- K-means ++

Algorithm 5.2 K-means++ initialization algorithm.

- 1: For the first centroid, pick one of the points at random.
 - 2: **for** $i = 1$ to *number of trials* **do**
 - 3: Compute the distance, $d(x)$, of each point to its closest centroid.
 - 4: Assign each point a probability proportional to each point's $d(x)^2$.
 - 5: Pick new centroid from the remaining points using the weighted probabilities.
 - 6: **end for**
-

Space and Time complexity of K-Means

Space Complexity:

Stores only:

- Data points
- Cluster centroids

Storage required:

$$O((m+K)n)$$

Where:

m = number of data points

n = number of attributes

K = number of clusters

Time Complexity

Time required:

$$O(I \times K \times m \times n)$$

Where:

I = number of iterations to converge

K-Means Additional Issues

- Handling empty clusters- **require special handling**
- Effect of outliers- **Outliers can distort centroids**
- Reducing SSE without increasing K- **SSE can be reduced using split–merge post processing- Incremental updates improve flexibility but add complexity**
- Updating centroids incrementally- **Practical K-means uses multiple heuristics for robustness**

K-Means Additional Issues- **Handling empty clusters**

- During assignment, no points are assigned to a centroid.
- Wastes a cluster, leads to higher SSE than necessary.

Strategies:

- Choose the point farthest from any centroid, this removes the point contributing most to SSE.
- Choose a replacement centroid from the cluster with highest SSE, or splits a large, poor-quality cluster.

K-Means Additional Issues- **Effect of Outliers**

- Squared error magnifies the influence of outliers
- Centroids shift away from dense regions SSE increases.
- Leads to Poor cluster representation.

Should outliers always be removed? - Not always.

- Examples where outliers matter: Financial analysis (high-value customers)
- Data compression (every point must be clustered)

Note: Decision depends on the application

K-Means Additional Issues- Reducing SSE Using Post processing

- Reduce SSE without increasing K
- K-means converges to a local minimum-(The total SSE cannot be reduced any further by small changes in cluster assignments or centroids, even though a better solution)
- Better clustering may exist
- Modify clusters after K-means converges

Use cluster splitting strategies -To reduce SSE by increasing clusters temporarily

- Split highest-SSE cluster
- Split high-variance cluster
- Introduce new centroid

K-Means Additional Issues- Reducing SSE Using Post processing

Use cluster merging strategies -to reduce cluster count while minimizing SSE increase

- Disperse a cluster
- Merge two clusters

K-Means Additional Issues- Incremental Updating of Centroids

- Standard K-means
 - Update centroids after all points are assigned
- Incremental K-means Update centroid after each point assignment

Advantages- Prevents empty clusters, Faster convergence and more flexible objective functions

Bisecting K-Means

- Bisecting K-means is an extension of the basic K-means algorithm
- Instead of forming all K clusters at once: It repeatedly splits clusters into two
- The idea is simple: keep splitting until K clusters are formed.

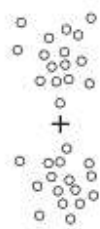
To obtain K clusters:

- Start with all data points in one cluster
- Split this cluster into two clusters using standard K-means ($K = 2$)
- Choose one of the existing clusters to split again
- Repeat the process until K clusters are produced
- At each step, the number of clusters increases by one

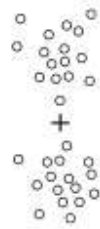
Bisecting K-Means

Algorithm 5.3 Bisecting K-means algorithm.

- 1: Initialize the list of clusters to contain the cluster consisting of all points.
 - 2: **repeat**
 - 3: Remove a cluster from the list of clusters.
 - 4: {Perform several “trial” bisections of the chosen cluster.}
 - 5: **for** $i = 1$ to *number of trials* **do**
 - 6: Bisect the selected cluster using basic K-means.
 - 7: **end for**
 - 8: Select the two clusters from the bisection with the lowest total SSE.
 - 9: Add these two clusters to the list of clusters.
 - 10: **until** The list of clusters contains K clusters.
-



(a) Iteration 1.



(b) Iteration 2.

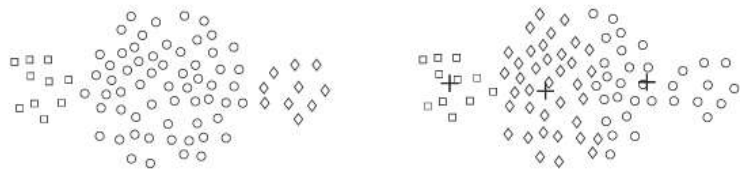


(c) Iteration 3.



K-Means and different types of Clusters

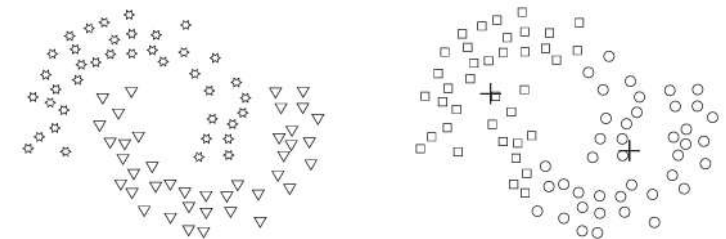
- K-means and its variations have a number of limitations with respect to finding different types of clusters.
- K-means has difficulty detecting the “natural” clusters, when clusters have non-spherical shapes or widely different sizes or densities.



(a) Original points.

(b) Three K-means clusters.

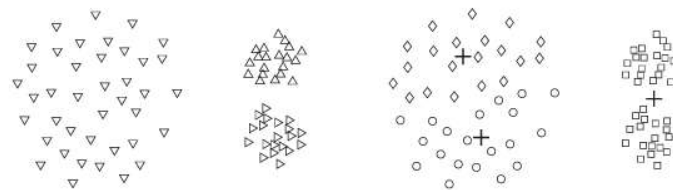
Figure 5.9. K-means with clusters of different size.



(a) Original points.

(b) Two K-means clusters.

Figure 5.11. K-means with non-globular clusters.



(a) Original points.

(b) Three K-means clusters.

Figure 5.10. K-means with clusters of different density.

K-means Strength and Weakness

- K-means is simple and can be used for a wide variety of data types.
- It is also quite efficient, even though multiple runs are often performed.
- Some variants, including bisecting K-means, are even more efficient, and are less susceptible to initialization problems.
- K-means is not suitable for all types of data.
 - It cannot handle non-globular clusters or clusters of different sizes and densities, although it can typically find pure subclusters if a large enough number of clusters is specified.
 - K-means also has trouble clustering data that contains outliers.
 - Outlier detection and removal can help significantly in such situations.
 - Finally, K-means is restricted to data for which there is a notion of a center (centroid).
 - A related technique, K-medoid clustering, does not have this restriction, but is more expensive.

K-means Strength and Weakness

- K-means is simple and can be used for a wide variety of data types.
- It is also quite efficient, even though multiple runs are often performed.
- Some variants, including bisecting K-means, are even more efficient, and are less susceptible to initialization problems.
- K-means is not suitable for all types of data.
 - It cannot handle non-globular clusters or clusters of different sizes and densities, although it can typically find pure subclusters if a large enough number of clusters is specified.
 - K-means also has trouble clustering data that contains outliers.
 - Outlier detection and removal can help significantly in such situations.
 - Finally, K-means is restricted to data for which there is a notion of a center (centroid).
 - A related technique, K-medoid clustering, does not have this restriction, but is more expensive.

Overview:

- In supervised classification, model evaluation is well defined(e.g., accuracy, cross-validation).
- In clustering, evaluation is less developed and less commonly used.
- Clustering is often exploratory, so evaluation may seem unnecessary.
- Different clustering algorithms define different types of clusters.
- Example:K-means → evaluated using SSE
- Density-based clusters → SSE is not suitable
- This diversity creates confusion about which evaluation measure to use.
- Cluster evaluation is crucial to: Verify cluster quality, Avoid false interpretations and ensure meaningful analysis

Cluster Validation issues:

1. Determining the clustering tendency of a set of data, i.e., distinguishing whether non-random structure actually exists in the data.
2. Determining the correct number of clusters.
3. Evaluating how well the results of a cluster analysis fit the data without reference to external information.
4. Comparing the results of a cluster analysis to externally known results, such as externally provided class labels.
5. Comparing two sets of clusters to determine which is better.

Unsupervised Cluster validity measure:

- Evaluate the quality of clustering using only the given data, without any external labels or ground truth.
- They answer the question: “How good is the clustering structure based on the data itself?”
- Example: SSE (Sum of Squared Errors), used commonly in K-means.
- These measures focus on two important aspects:
 - Cohesion (Compactness) – How close the data points are within the same cluster- Data points are tight and compact. SSE (lower value → better cohesion)
 - Separation (Isolation) – How far apart different clusters are. Minimal overlap between clusters.

Since they rely only on internal data properties, they are called internal indices.

Supervised Cluster validity measure:

- Supervised cluster validity measures evaluate how well the clusters formed by a clustering algorithm match an external or known structure, such as true class labels.
- These measures require external information that is not part of the dataset itself.
- They answer the question: “How closely do the discovered clusters correspond to the actual classes?”
- Example – Entropy: Measures the purity of clusters with respect to given class labels
- Lower entropy → clusters contain mostly one class → better clustering
- Because they rely on external labels, these measures are called external indices.

Relative Cluster validity measure:

- Relative cluster evaluation measures are used to compare two or more clustering results to decide which clustering is better.
- They do not introduce a new evaluation metric.
- Instead, they reuse existing supervised or unsupervised measures.
- The key idea is comparison, not absolute quality.
- Relative evaluation answers the question: “Which clustering is better among the given alternatives?”

Cluster Evaluation

Unsupervised Cluster Evaluation using Cohesion and Separation:

- Internal cluster validity measures evaluate clustering quality using only the dataset itself.
- For partitional clustering (e.g., K-means), most internal measures focus on:
 - Cohesion → how compact or tight the clusters are
 - Separation → how well clusters are isolated from each other
- These ideas apply to different clustering styles: Prototype-based clustering (e.g., K-means, centroids) or Graph-based clustering (e.g., similarity graphs)
- Even though the approaches differ, there are **interesting relationships** between their validity measures.
- Overall clustering quality is computed by: Evaluating each cluster individually, Combining these evaluations using a weighted sum
$$\text{overall validity} = \sum_{i=1}^K w_i \text{validity}(C_i).$$
- The weights may depend on: Equal importance (weight = 1) Cluster size More complex cluster properties (e.g., structure or connectivity)

Cluster Evaluation

Unsupervised Cluster Evaluation using Cohesion and Separation:

$$\text{overall validity} = \sum_{i=1}^K w_i \text{ validity}(C_i).$$

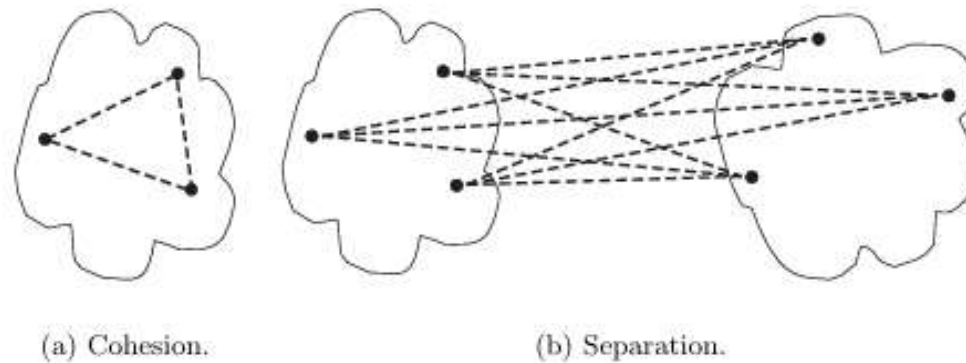


Figure 5.27. Graph-based view of cluster cohesion and separation.

Graph-Based View of Cohesion and Separation

- In graph-based clustering, data objects are represented as a proximity graph:
- Nodes → data points
- Edges (links) → connections between pairs of points
- Edge weights → **proximity between points(similarity or dissimilarity)**

Cohesion (Within a Cluster)

Cohesion measures **how strongly points inside the same cluster are connected**

It is defined as:

The **sum of the weights of all links connecting points within the cluster**

Higher internal connectivity → **better cohesion**

Separation (Between Clusters)

- Separation measures **how strongly two different clusters are connected**
- It is defined as:
 - The **sum of the weights of links connecting points in one cluster to points in another**
- Weaker connectivity between clusters → **better separation**

Role of Similarity vs Dissimilarity

If **similarity** is used as a proximity:

High internal similarity → good cohesion

Low cross-cluster similarity → good separation

If **dissimilarity (distance)** is used:

Low internal distance → good cohesion

High cross-cluster distance → good separation

$$cohesion(C_i) = \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_i}} proximity(\mathbf{x}, \mathbf{y}) \quad (5.9)$$

$$separation(C_i, C_j) = \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_j}} proximity(\mathbf{x}, \mathbf{y}) \quad (5.10)$$

Cluster Evaluation

Prototype-Based View of Cohesion and Separation

- In prototype-based clustering (such as K-means or K-medoids), each cluster is represented by a prototype:
- Centroid → mean of points (K-means)
- Medoid → most representative point (K-medoids)

Cohesion (Within a Cluster)

- The **sum of proximities (distances or similarities)** between each point and the cluster prototype
- If proximity is **squared Euclidean distance**, cohesion becomes: **SSE (Sum of Squared Errors)**

Separation (Between Clusters)

- Between cluster prototypes: Distance between centroids of two clusters.
- From overall prototype: Distance between a cluster centroid and the **global centroid** of the dataset

$$cohesion(C_i) = \sum_{\mathbf{x} \in C_i} proximity(\mathbf{x}, \mathbf{c}_i) \quad (5.11)$$

$$separation(C_i, C_j) = proximity(\mathbf{c}_i, \mathbf{c}_j) \quad (5.12)$$

$$separation(C_i) = proximity(\mathbf{c}_i, \mathbf{c}) \quad (5.13) \quad \mathbf{c} \text{ is the overall dataset centroid}$$

Cluster Evaluation

Relationship between Prototype-Based Cohesion and Graph-Based Cohesion

- **Graph-based view:** looks at **pairwise distances** (links) between all points in a cluster.
- **Prototype-based view:** looks at distances from points to a **single prototype** (centroid).
- However, **for certain proximity measures**, these two views become **mathematically equivalent**.

For Example: Euclidean distance & SSE

- In **Euclidean space**, when proximity is **squared Euclidean distance**:
 - The **Sum of Squared Errors (SSE)** (prototype-based) equals (up to a constant factor) the **average of all pairwise squared distances** within the cluster (graph-based).
- This means:
 - Measuring how far points are from the **centroid** or Measuring how far points are from **each other** gives the **same information about cohesion**.

$$\text{Cluster SSE} = \sum_{\mathbf{x} \in C_i} \text{dist}(\mathbf{c}_i, \mathbf{x})^2 = \frac{1}{2m_i} \sum_{\mathbf{x} \in C_i} \sum_{\mathbf{y} \in C_i} \text{dist}(\mathbf{x}, \mathbf{y})^2$$

C_i : the i -th cluster
 m_i : number of points in cluster C_i
 $\mathbf{c}_i$: centroid (mean) of cluster C_i

Cluster Evaluation

Relationship between Prototype-Based Separation measures

- For Euclidean distance, cluster separation is measured using Between-Group Sum of Squares (SSB)
- **SSB definition:**

$$\text{Total SSB} = \sum_{i=1}^K m_i \text{dist}(c_i, c)^2$$

- Measures how far each cluster centroid is from the overall mean. Higher SSB $\Rightarrow$ better separation
- If cluster sizes are equal:

$$\text{Total SSB} = \frac{1}{2K} \sum_{i=1}^K \sum_{j=1}^K \frac{m}{K} \text{dist}(c_i, c_j)^2$$

- The double sum considers all centroid pairs (c_i, c_j)
- The factor $1/2$ avoids double counting (since $\text{dist}(c_i, c_j)$ and $\text{dist}(c_j, c_i)$ are the same)
- The factor m/K accounts for equal cluster sizes
- The division by K normalizes over the number of clusters

Relationship between Cohesion and Separation

For **Euclidean distance**, three quantities are related:

1. **TSS (Total Sum of Squares)** → total variation in the data
2. **SSE (Sum of Squared Errors)** → measures **cohesion** (within-cluster compactness)
3. **SSB (Between-group Sum of Squares)** → measures **separation** (between-cluster spread)

$$\boxed{\text{TSS} = \text{SSE} + \text{SSB}}$$

TSS is constant for a given dataset

Reducing SSE automatically increases SSB

Therefore, **minimizing cohesion (SSE) is equivalent to maximizing separation (SSB)**

$$\begin{aligned}
 \text{TSS} &= \sum_{i=1}^K \sum_{x \in C_i} (x - c)^2 \\
 &= \sum_{i=1}^K \sum_{x \in C_i} ((x - c_i) - (c - c_i))^2 \\
 &= \sum_{i=1}^K \sum_{x \in C_i} (x - c_i)^2 - 2 \sum_{i=1}^K \sum_{x \in C_i} (x - c_i)(c - c_i) + \sum_{i=1}^K \sum_{x \in C_i} (c - c_i)^2 \\
 &= \sum_{i=1}^K \sum_{x \in C_i} (x - c_i)^2 + \sum_{i=1}^K \sum_{x \in C_i} (c - c_i)^2 \\
 &= \sum_{i=1}^K \sum_{x \in C_i} (x - c_i)^2 + \sum_{i=1}^K |C_i| (c - c_i)^2 \\
 &= \text{SSE} + \text{SSB}
 \end{aligned}$$

Cluster Evaluation

Overall Measures of Separation and Cohesion

- Cohesion (within-cluster similarity) and separation (between-cluster dissimilarity) were defined per cluster.
- To evaluate the entire clustering, these per-cluster values are combined using a weighted sum:

$$\text{Overall validity} = \sum_{i=1}^K w_i \cdot \text{validity}(C_i)$$

- The key design choice is what weights w_i to use.

Weights can vary widely:

$w_i=1$ → all clusters treated equally

$w_i=|C_i|$ → larger clusters get more importance

Some measures use **more complex weights** that depend on cluster structure.

- **CLUTO** is a clustering toolkit that uses the measures listed in **Table (next slide)**
- It uses **only similarity measures**: cosine, correlation, Jaccard, inverse Euclidean distance.
- Cohesion measures (I1, I2) Separation measures (E1) Combined cohesion + separation (G1)

Overall Measures of Separation and Cohesion

Name	Cluster Measure	Cluster Weight	Type
$\mathcal{I}_1$	$\sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_i}} sim(\mathbf{x}, \mathbf{y})$	$\frac{1}{m_i}$	graph-based cohesion
$\mathcal{I}_2$	$\sum_{\mathbf{x} \in C_i} sim(\mathbf{x}, \mathbf{c}_i)$		prototype-based cohesion
$\mathcal{I}_2$	$\sqrt{\sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_i}} sim(\mathbf{x}, \mathbf{y})}$		prototype-based cohesion
$\mathcal{E}_1$	$sim(\mathbf{c}_i, \mathbf{c})$	m_i	prototype-based separation
$\mathcal{E}_1$	$\sum_{j=1}^k \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_j}} sim(\mathbf{x}, \mathbf{y})$	$\frac{m_i}{\sqrt{(\sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_i}} sim(\mathbf{x}, \mathbf{y}))}}$	graph-based separation
$\mathcal{G}_1$	$\sum_{\substack{j=1 \\ j \neq i}}^k \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_j}} sim(\mathbf{x}, \mathbf{y})$	$\frac{1}{\sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_i}} sim(\mathbf{x}, \mathbf{y})}$	graph-based separation and cohesion

Evaluating Individual Clusters and Objects

- Cohesion and separation measures can be applied not only to the entire clustering, but also to individual clusters and even individual data objects.
- Clusters can be ranked based on their cohesion or separation values: High cohesion → better, more compact cluster, Low cohesion → candidate for splitting into sub clusters.
- Cluster refinement decisions can be guided by these measures:
 - Split clusters that are weakly cohesive
 - Merge clusters that are cohesive but poorly separated
- Individual objects can also be evaluated: Objects contributing strongly to cohesion/separation lie near the cluster interior
- Objects contributing weakly lie near the cluster boundary
- These ideas form the basis for advanced evaluation methods that assess: individual points, individual clusters and the overall clustering structure

Silhouette Coefficient

- The silhouette coefficient is a popular internal cluster validity measure because it combines both cohesion and separation into a single value.
- It evaluates how well each data point fits within its assigned cluster compared to other clusters.

Step 1: Compute cohesion (a_i)

- Calculate the **average distance** of point i to **all other points in its own cluster**
- This value is called a_i
- Smaller $a_i \rightarrow$ point is well placed inside its cluster

Step 2: Compute separation (b_i)

- For every **other cluster**, compute the **average distance** of point i to points in that cluster
- Choose the **minimum** of these values
- This value is called b_i
- Larger $b_i \rightarrow$ point is far from neighboring clusters

Step 3: Compute silhouette coefficient (s_i)

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Silhouette Coefficient

Interpretation of Silhouette Values

$s_i \approx +1 \rightarrow$ very good clustering

($a_i \approx 0$ and b_i is large)

$s_i \approx 0 \rightarrow$ point lies near the cluster boundary

$s_i < 0 \rightarrow$ poor clustering

(point is closer to another cluster than its own)

Effectively silhouette values to be positive and as close to 1 as possible

Silhouette Coefficient

- Cluster silhouette = average of silhouette values of all points in that cluster
- Overall silhouette coefficient = average silhouette of all data points
- Higher average silhouette → better overall clustering

- Unsupervised cluster evaluation methods that use the proximity (similarity/distance) matrix instead of only scalar measures like SSE or silhouette.

Two main ideas are covered:

- Quantitative evaluation using correlation between matrices
- Qualitative (visual) evaluation using plots of the similarity matrix

It also gives important limitations and cautions about unsupervised cluster validity.

Limitations:

- Many unsupervised validity measures exist, but: Most are designed for partitional, center-based clusters (e.g., K-means)
- They perform poorly for contiguity-based or density-based clusters
- A comparative study found: Most measures fail on arbitrary-shaped clusters
- CVNN (Clustering Validation index based on Nearest Neighbors) handles shapes better
- Silhouette performs well on many issues, except arbitrary shapes

Role of cohesion and separation in unsupervised measures

- Most unsupervised measures (e.g., silhouette): Combine cohesion (compactness) and separation
- As the number of clusters K increases- Measures improve until the natural clusters are found
- Then worsen when clusters are split too finely
- Hence, such measures can help estimate the number of clusters

NOTE:

- If the clustering algorithm's definition of a cluster differs from the evaluation measure's definition, results may disagree

Example: K-means optimizes SSE

- Silhouette optimizes cohesion + separation
- Optimal clusters according to SSE $\neq$ optimal clusters according to silhouette
- If data has sub clusters that are weakly separated, combined measures may give only a coarse view

Evaluating clustering using correlation of similarity matrices

Compare:

Actual similarity matrix (from data)

Ideal similarity matrix (from cluster labels)

Ideal similarity matrix

Similarity = **1** if two points are in the **same cluster**

Similarity = **0** otherwise

When reordered by cluster labels, it shows a **block-diagonal structure**

Evaluation

Compute **correlation** between:

actual similarity matrix

ideal similarity matrix

High correlation → good clustering

Low correlation → poor clustering

Correlation is computed only for the **upper or lower triangular part**

(because similarity matrices are symmetric)

Limitation

Not suitable for **density-based or contiguity-based clusters**

Such clusters:

are non-globular can be intertwined violate block-diagonal assumptions

Visual evaluation using similarity matrices

Instead of a single number, we can **visualize cluster structure**:

Procedure

- Reorder similarity matrix by cluster labels
- Plot the matrix

Interpretation

- **Strong block-diagonal pattern** → well-separated clusters
- **Weak or fuzzy blocks** → poor or artificial clusters

Summary

Unsupervised cluster evaluation:

- is inherently difficult
- depends heavily on cluster shape assumptions
- Scalar measures give **summaries**
- Proximity-matrix methods give **insight**

Best practice:

- combine numerical measures
- pairwise cohesion/separation analysis
- visual inspection

Determining Correct Number of cluster

- When we apply clustering (e.g., K-means), we must choose K, the number of clusters.
- Since clustering is unsupervised, K is not known in advance, so we rely on evaluation measures to guide us.
- Two commonly used measures: SSE (Sum of Squared Errors)
 - Measures cohesion (how compact clusters are)
 - As K increases, SSE always decreases
- Silhouette Coefficient- Combines cohesion and separation
 - Ranges from -1 to $+1$
 - Higher value $\rightarrow$ better clustering

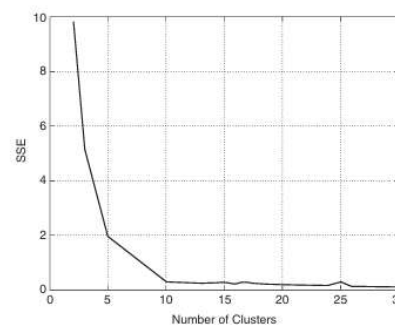


Figure 5.32. SSE versus number of clusters for the data of Figure 5.29 on page 364.

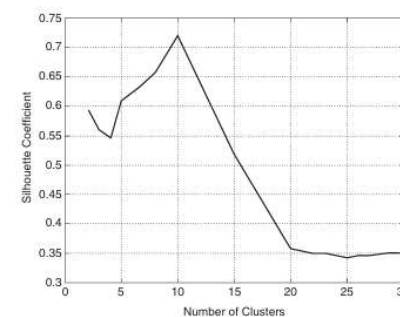


Figure 5.33. Average silhouette coefficient versus number of clusters for the data of Figure 5.29.

Supervised Measures of cluster validity

- When external information is available, it usually comes as class labels (ground truth).
- In such cases, clustering is evaluated by measuring how well cluster labels correspond to class labels.
- But why cluster if we already know the classes?

Clustering with external labels is useful because it helps to:

- Compare clustering algorithms with ground truth
- Check how well clustering recovers known class structure
- Automate manual classification
- Example: grouping news articles into topics automatically
- Support semi-supervised learning
- See whether objects in the same cluster tend to share the same label

- Two broad approaches to supervised cluster evaluation

(A) Classification-oriented measures

- Borrowed from classification evaluation
- Measure how pure or class-consistent each cluster is
- Examples: Entropy, Purity, Precision, Recall, F-measure

(B) Similarity-oriented measures

- Based on pairwise agreements
- Measure whether:
 - objects in the same class are placed in the same cluster
 - objects in different classes are separated

Example: Jaccard coefficient

These treat clustering as a pairwise similarity problem

Assume:

m_i : number of objects in cluster i

m_{ij} : number of objects in cluster i belonging to class j

$p_{ij} = m_{ij}/m_i$

Entropy

- Measures **class mixture** within a cluster
- Low entropy $\rightarrow$ cluster mostly contains one class
- High entropy $\rightarrow$ cluster contains many classes
- Overall entropy is a **cluster-size-weighted average**
 $\rightarrow$ **Entropy = 0 is ideal**

Purity

- Measures the dominant class proportion in a cluster
Purity of cluster i :
$$\text{purity}(i) = \max_j p_{ij}$$
- Overall purity is size-weighted
- Purity = 1 is ideal

Precision

- Of all objects in a cluster, how many belong to a given class?

$$\text{precision}(i, j) = p_{ij}$$

- Measures **cluster correctness**

Recall

- Of all objects of a given class, how many are captured by a cluster?

$$\text{recall}(i, j) = \frac{m_{ij}}{m_j}$$

- Measures **class coverage**

F-measure

- Combines precision and recall:

$$F(i, j) = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

- High only when **both precision and recall are high**

Summary

- Supervised cluster evaluation compares cluster labels with class labels
- Two types: Classification-oriented (entropy, purity, precision, recall, F)
- Similarity-oriented (pairwise agreement)
- Entropy and purity measure cluster homogeneity
- Precision and recall measure cluster–class correspondence
- F-measure balances precision and recall
- Confusion matrix gives the clearest insight

- Cluster validity measures quantify how good a clustering is, usually by producing a single numeric value.
- The main challenge is interpreting what that number actually means.
- Sometimes interpretation is easy: Purity = 1 $\rightarrow$ very good clustering, Purity = 0, Entropy high, or SSE high $\rightarrow$ poor clustering, Entropy = 0 or SSE = 0 $\rightarrow$ ideal clustering
- In many cases: There may be no clear minimum or maximum, The data scale can affect the value, Intermediate values are hard to judge
- If an absolute standard exists (e.g., maximum acceptable approximation error), interpretation is straight forward.
- Otherwise, clustering quality is often interpreted statistically:
 - Check whether the obtained validity value is unlikely to occur by random chance
 - A clustering is considered good if it reveals non-random structure in the data
- Valid clusters should produce unusually good validity scores compared to random data.

Example

- Suppose we cluster customer data using K-means and obtain:
- **SSE = 120**
- By itself, this number is hard to interpret, So we compare it with **randomly generated clusterings** of the same data:
- Random clusterings give SSE values around **300–400**
- Our clustering gives **SSE = 120**, which is **much lower**
- ☞ This large difference suggests that the clustering captures **real, non-random structure**, and the SSE value is therefore **meaningful and good**.

Choosing Cluster Validity measure

Overall Measures of Separation and Cohesion

- Many cluster validity measures exist; no single measure is universally best
- Choice depends on the goal of clustering:

Clustering for Summarization

- Goal: maximum compression of data
- Class labels not used
- Focus on minimizing representation error
- Objective: Minimize distance to nearest cluster centroid
- SSE is a natural and commonly used measure

Clustering for Understanding (Unsupervised)

- Goal: discover **structure and patterns**
- Most measures aim to: Maximize **cohesion**, Maximize **separation**
- Practical approach:
 - Examine cohesion of individual clusters
 - Examine pairwise separation between clusters
- Limitation:
 - Most measures do **not work well** for
 - density-based
 - contiguity-based
 - irregular or intertwined clusters

Choosing Cluster Validity measure

Overall Measures of Separation and Cohesion

- Many cluster validity measures exist; no single measure is universally best
- Choice depends on the goal of clustering:

Supervised Cluster Evaluation

Used when class labels are available

Goal: clusters should match underlying class structure

Evaluation is non-trivial

Common measures: F-measure, Hierarchical F-measure

When number of clusters is small: Confusion matrix is often more informative

Shows which classes mix across clusters

Supervised indices are: Independent of clustering type(centroid-based, density-based, or contiguity-based)